

Brute Force, Rainbow Tables & Password Hashes

Search-space math

```
charset 26 (a-z) ^ length 6 = 308,915,776 (~3e8)
alphabets+punct 95 ^ 8      = 6.6e15
```

Every extra char multiplies brutally. That's why length beats 'complexity'.

GPU economics

RTX-class: ~10 GH/s for MD5, ~100 Mil/s for bcrypt

MD5/ SHA1/NTLM cracks in hours-minutes with rule lists

bcrypt/argon2 with cost: seconds-to-years per hash → chosen to make GPUs sad

Salt, the equalizer

Salt random-per-user: no precomputed tables (rainbow) reuse; identical passwords hash differently

Slow hash (work factor) multiplies per-attempt cost

Bottom line: salted+slow changes attack from 'instantly look up' to 'brute force per user'

Rainbow tables (and why they lost)

Precomputed hash→plaintext chain tables (1 TB of chains). Speed: $O(1)$ lookups. They died to: salts (make tables useless) + modern memory. They still exist to explain *time-memory tradeoffs*.

The practical crack ladder

```
# from your OWN hashes only
hashcat -m 0 -a 0 hash.txt rockyou.txt           # fast
hashcat -m 1000 -a 0 hash.txt rockyou.txt -r rules/best64.rule
hashcat -m 3200 -a 3 '?l?l?l?l?d?d?'           # bcrypt mask (slow!)
john --format=raw-sha256 --wordlist=rockyou.txt hash.txt
```

Defense takeaways

Store with argon2/bcrypt + per-user salt + reasonable cost

Enforce length (15+) over mere 'complex'

Never reuse; breach-compare; no plaintext heaven (the wiki of 'pwned passwords')

[← Cryptographic Misuse Patterns](#)

[Timing Attacks & Side Channels...](#)